

Bottleneck — Livrable 1 : Veille métier et technologique

Amélioration du livrable Projet 6 augmentée par l'IA

Zied Belhocine

1. Besoins de veille identifiés

Après relecture du projet et de mes livrables, j'ai identifiés deux axes d'améliorations du projets :

#	Besoin de veille	Origine du besoin
1	Améliorer la restitution visuelle (dataviz) du notebook, pour un usage exploitable par les équipes commerciales (drill-down catégorie → produit), en comparant Plotly et Altair	Le notebook initial repose sur des graphiques statiques. Pour comparer les ventes dans chaque catégorie, il faut produire autant de graphiques que de catégories.
2	Créer un script d'automatisation du nettoyage et de la fusion des données (ERP + web + liaison), en comparant deux assistants IA — Claude et GitHub Copilot (VS Code)	Le nettoyage/fusion est réalisé manuellement, cellule par cellule dans le notebook ce qui est fastidieux. Un script automatique permettrait de gagner en temps et lisibilité.

2. Panel de solutions comparées

2.1 Besoin 1 — Dataviz (catégorie : outils/librairies)

- **Option A** : Plotly est l'une des seules bibliothèques spécifiquement tournées vers les représentations interactives.
- **Option B** : Altair est une bibliothèque Python déclarative de visualisation de données statistiques.

2.2 Besoin 2 — Automatisation du nettoyage/fusion (catégorie : approches IA)

- **Option A** : Claude (Anthropic), chat via page web.
 - **Option B** : GitHub Copilot (VS Code), sollicité avec un contexte de prompt équivalent mais directement à l'intérieur de mon répertoire, lui donnant accès à tous mes fichiers.
-

3. Sources fiables identifiées

Solution	Source	Lien	Date de consultation
Plotly (graph_objects, update_menus)	Documentation officielle	https://plotly.com/python/custom-buttons/	04/08/2026
Plotly, référence technique complète	Documentation officielle	https://plotly.com/python/	13/08/2026
Altair / Vega-Lite (selection_point, transform_filter)	Documentation officielle Vega-Altair	https://altair-viz.github.io/user-guide/transform/filter.html	13/08/2026
Altair, page d'accueil et vue d'ensemble	Documentation officielle Vega-Altair (v6.2.2)	https://altair-viz.github.io/	13/08/2026
Comparatif Plotly / Altair / Matplotlib / Seaborn (blog francophone)	Liora (ex-DataScientest), institut de formation data francophone	https://liora.io/comparatif-bibliotques-pyton-dataviz	04/08/2026
« Altair : tout savoir sur cette bibliothèque » (article francophone)	Liora (ex-DataScientest), auteur Jérémie Robert	https://liora.io/altair-tout-savoir	13/08/2026

Solution	Source	Lien	Date de consultation
« Réaliser de la data visualisation grâce à Plotly » (article francophone)	Liora (ex-DataScientest)	https://liora.io/réaliser-de-la-data-visualisation-grâce-a-plotly	04/08/2026
« Créer des visualisations interactives avec Plotly » (leçon francophone)	Programming Historian, traduction française (Axel Morin)	https://programminghistorian.org/fr/lecons/visualisations-interactives-plotly	13/08/2026
« Construire des graphiques avec Python » — chapitre comparant Plotly et Altair sur un même cas d'usage	Cours ENSAE « Python pour la data science », Lino Galiana	https://python.ds.linogaliana.fr/content/visualisation/matplotlib.html	13/08/2026
« Altair Tutorial: Create Pro Data Visualizations in 3 Lines of Python Code » (vidéo, fonctionnalités de base)	YouTube	https://www.youtube.com/watch?v=CiYBkbbECpI	13/08/2026
GitHub Copilot dans VS Code	Documentation officielle GitHub (fr)	https://docs.github.com/fr/copilot?tool=vscode	13/08/2026
Claude (Anthropic)	Documentation officielle	https://docs.claude.com	13/08/2026
« Bugs in Large Language Models Generated Code: An Empirical Study » — taxonomie académique des types de bugs	Tambon et al., Polytechnique Montréal (financée par FRQ, CIFAR, CRSNG)	https://arxiv.org/html/2403.08937v1	13/08/2026

Solution	Source	Lien	Date de consultation
dans le code généré par IA			
Claude vs GitHub Copilot, comparatif 2026 (blog francophone)	eesel AI	https://www.eesel.ai/fr/blog/claude-vs-copilot-comparaison	13/08/2026

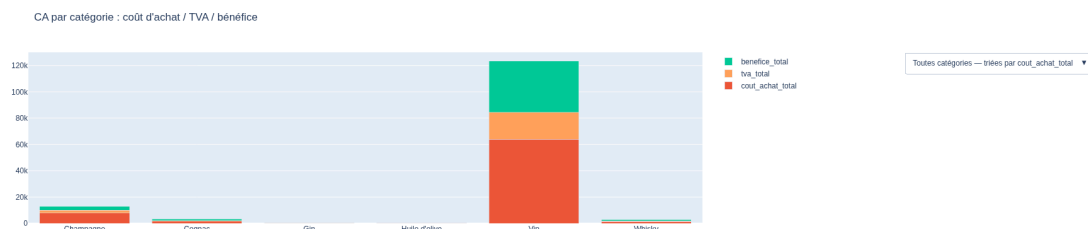
Note méthodologique : pour le besoin 2, la source la plus significative n'est pas documentaire mais **expérimentale**. En effet, je n'ai pas trouvé d'articles comparant les deux outils sur les tâches à accomplir. J'ai donc comparé leur efficacité directement sur le projet.

Précision sur la source eesel AI : cet article (et la majorité des comparatifs disponibles en ligne) compare surtout **Claude Code** (l'agent CLI d'Anthropic) à Copilot, pas Claude en dialogue conversationnel classique tel qu'utilisé dans ce projet. La distinction est réelle : Claude Code est un agent autonome de terminal, tandis que ce projet a utilisé Claude en échange itératif (chat). L'article reste utile pour la partie contextuelle (modèles, tarifs, positionnement général), mais pas comme preuve d'équivalence stricte avec notre usage.

4. Comparaison selon critères explicites

4.1 Besoin 1 — Plotly vs Altair (test réel)

Test réalisé : les deux librairies ont été utilisées pour produire la même visualisation (barplot empilé coût d'achat / TVA / bénéfice, avec filtre interactif par catégorie et drill-down vers le détail produit) sur `df_merge`.



Barplot empilé Plotly fonctionnel : CA par catégorie, décomposé en coût d'achat / TVA / bénéfice

Critère	Plotly (graph_objects + update_menus)	Altair (selection_point + transform_filter)
Qualité	Lisible même avec le drill-down produit activé	Ne permet pas de Drill-down mais peut être filtré par catégorie pour n'observer que certains produits.
Robustesse / biais	Aucun problème observé lors du test	Aucun bug bloquant, mais nécessite des données strictement au format long (retraitement systématique par un df.melt)
Coût temps	Lecture de documentation mais utilisation simple car proche de matplotlib et seaborn	Réécriture complète nécessaire (reformatage en long, nouvelle syntaxe de sélection)
Reproductibilité	Élevée	Élevée également
Sécurité / conformité	RAS	RAS
Maintenabilité	Le drill-down catégorie → produit reste exploitable en l'état pour un usage commercial	Filtrage combiné (catégorie + tri) non résolu ;

Limite technique observée côté Plotly, pour être complet : un vrai croisement de deux filtres indépendants (catégorie *et* tri, sélectionnables séparément) n'est pas possible nativement avec Plotly Express/graph_objects seul — chaque combinaison doit être précalculée. Une vraie solution à filtres croisés nécessiterait **Dash**, identifié comme piste d'évolution future plutôt que implémenté dans cette itération.

Décision retenue, fondée sur le test : **Plotly est conservé** pour la restitution destinée aux équipes commerciales : le drill-down catégorie → produit — me semble particulièrement utile pour un usage commercial — fonctionne avec Plotly alors qu'il est impossible sur Altair et les visualisations peuvent y devenir illisibles s'il y a trop d'éléments .

4.2 Besoin 2 — Claude vs GitHub Copilot (test réel)

Test réalisé : un même besoin (script Python réutilisable de consolidation ERP/web/liaison, avec rapport d'anomalies) a été soumis aux deux assistants, à partir d'un contexte de prompt équivalent (schéma des 3 fichiers, règles de nettoyage attendues, objectif de réutilisabilité mensuelle). Chaque script a ensuite été exécuté sur les données réelles du projet Bottleneck.

Chronologie du test :

1. Les deux scripts ont été générés séparément, avec des architectures différentes : Claude a produit une fonction importable retournant les données en mémoire (pensée pour un usage direct dans le notebook) ; Copilot a produit un script CLI autonome écrivant des fichiers sur disque (.xlsx, .csv, .json), avant d'ajouter sur demande un wrapper notebook (notebook_api.py) permettant d'appeler la fonction dans un notebook.
2. **Les deux scripts, exécutés indépendamment sur les mêmes données réelles, produisent un résultat strictement identique : 712 lignes consolidées**, avec correspondance exacte sur chaque métrique intermédiaire (prix négatifs : 3, stocks négatifs corrigés : 2, hors vente exclus : 106, doublons ERP : 0, sku manquants : 85, non-produit exclus : 714, product_type manquant : 1, doublons web : 0).

Critère	Claude	GitHub Copilot
Qualité	Dataframe de sortie plus concis sur les colonnes utiles au notebook (déduite de la lecture complète du notebook original) ;	Sortie plus large (conserve aussi stock_status, tax_status) ;
Robustesse / biais	Pas de garde-fou sur fichiers manquants ;	Vérifie l'existence des fichiers, coercion numérique défensive ;
Coût temps	Script plus court (~130 lignes), conçu directement pour l'usage réel (notebook)	Script deux fois plus long (~270 lignes), incluant une gestion CLI et une détection automatique de clé de jointure pour des colonnes hypothétiques qui n'existent pas dans ce schéma — un peu de sur-ingénierie par rapport au besoin exact
Reproductibilité	Retour en mémoire, pensé pour continuer l'analyse dans le même notebook	Écrit un fichier anomalies_report.json sur disque à chaque exécution — artefact persistant, plus robuste comme preuve dans la

Critère	Claude	GitHub Copilot durée qu'un simple print()
Sécurité / conformité	RAS	RAS
Maintenabilité	Plus simple, cohérent avec le reste du notebook existant	Type hints, logging, argparse — plus proche des standards "production" ; wrapper notebook ajouté ensuite pour combler l'écart d'usage

Décision retenue, fondée sur le test : Je prends la décision de conserver le script de Claude car il est plus simple, lisible et donc plus facile à entretenir, améliorer et déboguer.

5. Piste d'amélioration continue et automatisation de la veille

Amélioration continue : pour le besoin 1, industrialiser un dashboard Plotly/Dash pour lever la limite du filtrage combiné identifiée en section 4.1. Pour le besoin 2, poursuivre l'usage comparatif de plusieurs assistants IA sur les prochaines évolutions du script, plutôt que de se reposer sur un seul outil sans vérification croisée.

Élément d'automatisation : abonnement possible aux flux RSS de releases des dépôts GitHub des bibliothèques clés (plotly, altair) via <https://github.com/<org>/<repo>/releases.atom>, pour être notifié des évolutions susceptibles de changer les résultats de cette comparaison (ex. une future version d'Altair simplifiant le filtrage combiné).

Besoins en formation identifiés : Formation aux librairies plotly (express, graph_objects et Dash) serait un vrai plus pour le data Analyst que je suis (même si l'IA peut faire le job) afin de produire des visualisations en autonomie et corriger un code erroné.